

Relatório Técnico Completo

Pipeline de Big Data: Do Processamento Bare-Metal à Visualização

1. Introdução e Escopo

Este relatório documenta a implementação de um pipeline de dados ponta a ponta para diagnóstico de resíduos sólidos. O projeto foi arquitetado em nuvem (OCI), com infraestrutura customizada, processamento distribuído e orquestração de containers para garantir escalabilidade industrial.

2. Infraestrutura: Kubernetes "Do Zero" (Bare-Metal)

Diferente de soluções de abstração (como *Kind* ou *Minikube*), o cluster foi construído de forma granular:

- **Instalação Manual:** Utilização dos binários `kubeadm`, `kubelet` e `kubectl`.
- **Runtime nativo:** Configuração do **Containerd** como engine de execução, garantindo conformidade com padrões de mercado e alta performance.
- **Rede e Segurança:** Configuração manual do `iptables` e das *Security Lists* da Oracle Cloud para exposição controlada de serviços.

3. Stack Tecnológica Completa

- **Cloud:** Oracle Cloud Infrastructure (Free Tier).
- **Orquestração:** Kubernetes (K8s) via `kubeadm`.
- **Processamento Big Data:** Apache Spark (PySpark) executando via Spark-on-K8s Operator.
- **Container Runtime:** Containerd (Docker-free environment).
- **Visualização:** Streamlit para dashboards interativos.
- **MLOps & Futuro:** Estrutura pronta para integração com **Kubeflow**.
- **Versionamento:** Git/GitHub com fluxos de CI documentados.

4. Etapas do Pipeline e Sucessos

Processamento (ETL)

Execução de jobs Spark para limpeza, agregação e cálculo de toneladas de resíduos em grandes volumes de dados. A persistência foi garantida via **Persistent Volume Claims (PVC)**, permitindo o compartilhamento de dados entre os pods de processamento e o dashboard.

Exposição Pública

Implementação de serviço público via `kubectl port-forward`, estabilizado com o comando `nohup` para persistência pós-sessão SSH. O sucesso foi a transição de um projeto local para um ambiente online acessível globalmente.

5. Desafios Enfrentados

- **Rede OCI:** Conflitos entre firewall da nuvem e regras de rede interna do sistema operacional.
- **Gestão de Processos:** Necessidade de limpar portas bloqueadas (`fuser`) e manter serviços em background (`nohup`).
- **Limitações de Hardware:** Otimização extrema de memória para rodar Spark dentro dos limites de uma VM Free Tier.
- **Autenticação:** Transição para tokens de segurança (PAT) para versionamento no GitHub.

6. Escalabilidade e Visão de Engenharia

Apesar de o resultado final ser um dashboard, o sistema foi desenhado para escalonamento horizontal:

- **Spark-on-K8s:** Preparado para escalar executores conforme a carga cresce.
- **Desacoplamento:** Cada camada (armazenamento, processamento, UI) atua como um microserviço independente.
- **Pronto para Kubeflow:** A estrutura permite que o pipeline atual evolua para uma plataforma completa de MLOps.

